

Online Test Platformu PRD ve Yol Haritasi

Icindekiler

1. Proje Ozeti
2. Ekip ve Sorumluluk Yaklasimi
3. Urun Hedefleri
4. Kullanici Roller
5. Temel Ekranlar
6. Netlestirilmis Teknik Yapı
7. Netlestirilmis Veritabani Modelleri
8. Fazlara Bolunmus Yol Haritasi
9. Arda Icin Net Gorev Sahipligi
10. Emir Icin Net Gorev Sahipligi
11. Onerilen Haftalik Calisma Plani
12. MVP Kapsami (Sayfa ve Akis Bazinda)
13. Verilmis Kararlar
14. Basari Kriterleri
15. Dosya ve Kod Organizasyonu Kurallari
16. Ilk Yapilacaklar
17. Arastirma Kaynaklari
18. Gizlilik, KVKK ve Hukuki Notlar
19. Hata Yonetimi, Loglama ve Izleme
20. SEO ve Metadata
21. Erisilebilirlik ve Tarayici Destegi
22. Test Guvenligi
23. Icerik Sayfalari
24. Backend Alternatifleri (Arastirma Ozeti)

1. Proje Ozeti

Bu proje, ogrencilerin ders secerek online test cozebildigi, hocalarin ise soru havuzu, test yonetimi ve ogrenci takibi yapabildigi bir online sinav platformudur.

Platform iki ana kullanici grubuna hizmet eder:

- Ogrenciler: Ana sayfadan online teste girer, ders secer, test cozer ve sonucunu gorur.
- Hocalar: Uyelik/giris sistemiyle panele girer, kendisine tanimli ogrencileri gorur, soru yukler, soru havuzundan test olusturur ve sinava giren ogrencileri takip eder.

Projede ayrıca reklam alanları bulunacaktır. Reklam bölümleri ilk etapta statik alanlar olarak başlayacak, sonraki fazlarda panelden yönetilebilir hale getirilecektir.

Bu doküman projenin ana karar kaynağıdır. Uygulamada yapılacak değişiklikler önce bu dosyaya eklenmeli, sonra kodlanmalıdır.

2. Ekip ve Sorumluluk Yaklaşımı

Projeyi iki kişi geliştirecek:

- Arda
- Emir

İş bölümü alan sahipliğine göre ayrılır. Her kişi kendi alanındaki ekran, component veya teknik akışın tamamlanmasından sorumludur.

Arda'nın ana sorumluluk alanı:

- Proje kurulumu ve mimari kararlar
- Veritabanı tasarımı
- Auth ve rol kontrolü
- Öğrenci test akışının iş mantığı
- Test sonuç hesaplama ve kayıt
- Hoca panelindeki veri akışları
- Soru havuzu ve test oluşturma backend mantığı
- Deployment ve teknik kontroller

Emir'in ana sorumluluk alanı:

- Ana sayfa arayüzü
- Header, footer ve ortak görsel bölümler
- Ders kartları ve ders seçim sayfası arayüzü
- Öğrenci bilgi formu arayüzü
- Test başlangıç ve test çözüm ekranlarının görsel düzeni
- Sonuç ekranı arayüzü
- Hoca paneli dashboard kartları
- Liste, tablo ve boş durum ekranları
- Reklam alanları
- Demo içerik ve sayfa metinleri

Ortak çalışma kuralları:

- Her faz sonunda çalışan bir ekran veya akış teslim edilir.
- Arda teknik entegrasyonu ve veri bağlantılarını kontrol eder.
- Emir arayüz, içerik ve responsive görünümü kontrol eder.
- Bir görev baska kişinin alanını etkiliyorsa değişiklik önce bu dokümana eklenir.

3. Urun Hedefleri

Ana Hedefler

- Ogrencilerin kolayca ders secip test cozmesini saglamak.
- Hocalarin soru yukleyip soru havuzundan test olusturabilmesini saglamak.
- Hocalarin sinava giren ogrencileri ve sonuclarini gorebilmesini saglamak.
- Hocalara kendisine tanimli ogrenciler uzerinden takip imkani vermek.
- Reklam alanlariyla platformda gelir modeli icin temel hazirlamak.

Ilk Surumde Odak

Ilk surumda amac, urunun calisan bir MVP halini cikarmaktir:

- Ana sayfa
- Ogrenci icin ders secimi
- Test cozme ekranı
- Test sonucu kaydı
- Hoca girişi
- Hoca paneli
- Soru havuzu
- Test oluşturma
- Sinava giren ogrencileri görüntüleme
- Temel reklam alanları

4. Kullanici Rollerı

4.1 Ziyaretci / Ogrenci

Ogrenci ilk etapta uyelik olmadan teste girebilir.

Ogrenci akisi:

1. Ana sayfaya girer.
2. "Online Test" butonuna basar.
3. Dersleri gorur.
4. Bir ders secer.
5. O derse ait aktif testi cozer.
6. Testi bitirir.
7. Sonucunu gorur.

Ogrenciden alinabilecek bilgiler:

- Ad soyad

- Telefon veya e-posta
- Sinif seviyesi
- Bagli oldugu hoca, varsa

Not: "Herhangi biri girebilecek" istegi nedeniyle ogrenci girisi zorunlu olmayacaktır. Ancak sonuc takibi yapabilmek icin test baslamadan once minimum kimlik bilgisi alınmalidir.

Kimlik dogrulama:

- Ogrenci bilgi formunda girilen telefon veya e-posta adresine tek kullanimlik dogrulama kodu (OTP) gönderilir.
- OTP dogrulanmadan test baslatilamaz.
- Bu adim sahte bilgi girilerek tekrar deneme engelinin bypass edilmesini onler.
- OTP göndermek icin e-posta servisi (Resend) kullanılacaktır. SMS entegrasyonu ileri faza birakilmistir.
- OTP kodlari 5 dakika gecерlidir ve tek kullanimlidir.

4.2 Hoca

Hoca uyelik/giris sistemiyle panele girer.

Hoca yetenekleri:

- Kendi paneline giris yapar.
- Kendisine tanimli ogrencileri gorur.
- Sinava giren ogrencileri gorur.
- Ogrenci sonuclarini inceler.
- Soru yukler.
- Soru havuzunu gorur.
- Soru havuzundan test olusturur.
- Testleri aktif/pasif yapar.

4.3 Admin

Admin rolu ilk etapta temel kapsamda tutulabilir ama sistemin saglikli ilerlemesi icin gereklidir.

Admin yetenekleri:

- Hoca hesaplarini yonetir.
- Hocalara ogrenci tanimlar.
- Dersleri yonetir.
- Reklam alanlarini ilerleyen fazlarda yonetir.
- Gerekirse tum soru havuzunu ve testleri denetler.

5. Temel Ekranlar

5.1 Ana Sayfa

Icerik:

- Platform tanitimi
- "Online Teste Basla" butonu
- Derslere yonlendirme
- Hocalar icin giris butonu
- Reklam alanı
- Temel bilgilendirme bolumleri

Sorumluluk:

- Emir: Tasarım, statik icerik, reklam alanlarinin yerlestirilmesi.
- Arda: Online test ve hoca paneli yonlendirme mantigi.

5.2 Ders Secim Sayfasi

Icerik:

- Ders kartlari
- Her ders icin ad, aciklama ve test sayisi
- Aktif testi olmayan derslerde uygun bilgilendirme

Sorumluluk:

- Emir: Ders kartlari ve sayfa duzeni.
- Arda: Ders verilerinin veritabanindan cekilmesi.

5.3 Test Baslangic Sayfasi

Icerik:

- Secilen dersin adi
- Test kurallari
- Ogrenci bilgi formu
- "Teste Basla" butonu

Sorumluluk:

- Emir: Form arayuzu ve aciklama metinleri.
- Arda: Form verisinin dogrulanmasi ve test oturumu baslatma.

5.4 Test Cozme Ekrani

Icerik:

- Soru metni
- Siklar
- Sonraki / onceki soru
- Cevap isaretleme

- Sure varsa sure gostergesi
- Testi bitir butonu

Sorumluluk:

- Arda: Test motoru, cevap kaydi, sure, sonuc hesaplama.
- Emir: Soru karti arayuzu ve gorsel iyilestirmeler.

5.5 Sonuc Ekrani

Icerik:

- Dogru sayisi
- Yanlis sayisi
- Bos sayisi
- Puan
- Ders bilgisi
- Soru bazli cevap ozeti (ogrencinin secimi ve dogru cevap)

Sorumluluk:

- Arda: Sonuc hesaplama ve kaydetme.
- Emir: Sonuc ekraninin tasarimi.

5.6 Hoca Girisi

Icerik:

- E-posta ve sifre giris formu
- Hatali giris uyarisi
- Sifre sifirlama akisi (Faz 9'da eklenecek)

Sorumluluk:

- Arda: Auth, rol kontrolu, guvenlik.
- Emir: Form tasarimi.

Notlar:

- Hocalar kendileri kayit olamaz; hesaplari admin tanimlar (bkz. Bolum 13).
- Public bir kayit sayfasi MVP'de olmayacaktır.

5.7 Hoca Paneli

Icerik:

- Ozet dashboard
- Toplam ogrenci
- Sinava giren ogrenciler

- Son test sonuclari
- Hizli aksiyonlar

Sorumluluk:

- Arda: Panel veri mantigi.
- Emir: Dashboard kartlari ve temel gorsel duzen.

5.8 Soru Havuzu

Icerik:

- Sorulari listeleme
- Derse gore filtreleme
- Zorluk seviyesine gore filtreleme
- Soru ekleme
- Soru duzenleme
- Soru pasife alma (soft-delete)

Sorumluluk:

- Arda: Veritabani, CRUD, validasyon, yetki kontrolu.
- Emir: Liste arayuzu, filtre alanlari ve form tasarimi.

Notlar:

- Veri butunlugu icin sorular kalici olarak silinmez.
- `is_active = false` ile pasife alinir.
- Bir teste ekli sorular pasife alinsa bile testlerde gorunmeye devam eder; sadece havuzda yeni eklemelerde listelenmez.

5.9 Test Olusturma

Icerik:

- Test adi
- Ders secimi
- Soru havuzundan soru secimi
- Testi aktif/pasif yapma
- Test ayarlari (sure, anonim katilim izni, sonuc gosterimi)

Notlar:

- Hoca yalnızca kendi sorularini teste ekleyebilir (bkz. Bolum 13). Ortak soru havuzu ileri fazda dusunulebilir.

Sorumluluk:

- Arda: Test olusturma mantigi, soru-test iliskisi, aktif test kontrolu.
- Emir: Form tasarimi ve secim arayuzu.

5.10 Sinava Giren Ogrenciler

Bu sayfa, **hocanın kendi olusturdugu testlere giren** ogrencileri listeler. Her hoca yalnızca kendi testlerine ait sonuclari gorur. Tum sistem genelinde sinava giren ogrencileri sadece admin gorebilir.

Icerik:

- Ogrenci adi
- Test adi
- Ders
- Puan
- Dogru/yanlis/bos sayilari
- Sinav tarihi
- Detay görüntüleme

Sorumluluk:

- Arda: Liste sorgulari, filtreleme, yetki kontrolu.
- Emir: Tablo/list UI ve bos durum ekranlari.

5.11 Reklam Bolumu

Ilk faz:

- Ana sayfada reklam alanı
- Ders secim sayfasında reklam alanı
- Sonuc ekranında reklam alanı

Ilk faz:

- Admin panelden reklam ekleme
- Reklam aktif/pasif yapma
- Baslangic ve bitis tarihi
- Tiklanma takipleri

Sorumluluk:

- Emir: Statik reklam alanlari ve gorsel yerlestirme.
- Arda: Ilk fazdaki reklam yonetimi ve veritabani mantigi.

6. Netlestirilmis Teknik Yapi

Internet arastirmasi ve proje kapsamından sonra MVP için tek ana teknoloji yolu secilmistir. Bu projede ayrı bir backend uygulaması kurulmayacak; frontend, server-side işlemler, API ihtiyaclari, auth entegrasyonu ve veritabanı erisimi Next.js içinde yönetilecektir.

6.1 Ana Teknoloji Stack'i

- Framework: Next.js App Router
- Dil: TypeScript
- UI: Tailwind CSS + shadcn/ui
- Form ve validasyon: React Hook Form + Zod
- Tablo/listeleme: TanStack Table
- ORM: Prisma
- Veritabani: PostgreSQL
- Veritabani hosting: Neon
- Auth: Auth.js (NextAuth)
- Yetkilendirme: Uygulama seviyesinde RBAC + server-side ownership kontrolu
- Dosya/gorsel saklama: Vercel Blob, ilerleyen fazda
- Deployment: Vercel
- Versiyon kontrol: Git + GitHub

6.2 Neden Bu Stack?

- Next.js App Router, public sayfalar, ogrenci test akisi, hoca paneli ve admin panelini ayni proje icinde duzenli ayirmaya uygundur.
- Prisma, PostgreSQL ile tip guvenli veritabani erisimi ve migration yonetimi saglar.
- PostgreSQL, hoca-ogrenci-test-sonuc gibi iliskisel veri yapilari icin uygundur.
- Auth.js (NextAuth), Next.js icinde hoca/admin girisi, session yonetimi ve rol kontrolu icin kullanilir.
- shadcn/ui component kodunu proje icine aldigi icin dashboard, form, tablo ve dialog gibi parcalar kolayca ozellestirilebilir.
- Vercel, Next.js projeleri icin hizli preview ve production deployment saglar.

6.3 Next.js Mimari Kararlari

Uygulama `src` klasoru altinda gelistirilecektir.

Onerilen route gruplari:

- `src/app/(public)` : Ana sayfa, ders secim sayfasi, reklamli public alanlar.
- `src/app/(student)` : Test baslangic, test cozme ve sonuc ekranlari.
- `src/app/(teacher)` : Hoca girisi sonrasi panel sayfalari.
- `src/app/(admin)` : Admin tarafindan kullanilacak yonetim sayfalari.
- `src/app/api` : Gerekli route handler endpointleri.

Temel route yapisi:

- `/` : Ana sayfa
- `/online-test` : Ders secim sayfasi
- `/online-test/[courseId]` : Dersin test baslangic sayfasi

- `/test/[testId]/start` : Öğrenci bilgi formu ve test kuralları
- `/test/[attemptId]` : Test çözme ekranı
- `/test/[attemptId]/result` : Sonuç ekranı
- `/teacher/login` : Hoca girişi
- `/teacher/dashboard` : Hoca panel ana sayfası
- `/teacher/questions` : Soru havuzu
- `/teacher/tests` : Test yönetimi
- `/teacher/students` : Kendisine tanımlı öğrenciler
- `/teacher/results` : Sınava giren öğrenciler ve sonuçlar
- `/admin` : Admin paneli, ileri faz

6.4 Backend ve Veritabanı Kararları

Backend tarafında:

- Veritabanı erişimi Prisma üzerinden yapılır.
- Veritabanı seması `prisma/schema.prisma` dosyasında tutulur.
- Migration'lar Prisma Migrate ile yönetilir.
- Kullanıcı, rol, session ve hesap tabloları Auth.js Prisma adaptörü ile yönetilir.
- Uygulama kullanıcı detayları `profiles` tablosunda tutulur.
- Hoca ve admin rolleri `profiles.role` alanıyla başlatılır.
- Hoca paneli verilerinde `teacher_id`, `owner_teacher_id` ve `teacher_students` eşleştirme tablosu üzerinden ownership kontrolü yapılır.
- Client tarafından doğrudan veritabanı erişimi olmayacak; veritabanı işlemleri server action veya route handler içinden yapılacaktır.

Server Action ve Route Handler kullanım kuralı:

- Mutations (veri oluşturma, güncelleme, silme): Server Actions kullanılır.
- Üçüncü parti webhook veya dışarıdan gelen HTTP istekleri: Route Handlers kullanılır.
- Veri okuma (read): Server component içinde doğrudan Prisma sorgusu veya Server Action kullanılır.

Connection pooling:

- Neon veritabanına bağlanırken `-pooler` suffix'li connection string kullanılacaktır.
- `@prisma/adapters-neon` paketi kurularak Vercel serverless ortamında her invocation'da yeni bağlantı açılması önleneyecektir.
- `DATABASE_URL` değişkeni pooler endpoint'ini, `DIRECT_URL` değişkeni doğrudan bağlantı endpoint'ini gösterecektir.
- Prisma schema'da `directUrl` ayarı migration işlemleri için kullanılacaktır.

İlk MVP için roller:

- `admin`

- `teacher`

Ogrenci ilk etapta zorunlu uyelik olmadan test cozer. Ogrencinin girdigi bilgiler `students` tablosuna kaydedilir ve test denemesiyle iliskilendirilir.

6.5 Guvenlik ve Yetkilendirme

Guvenlik kurallari:

- Hoca paneli giris yapmayan kullaniciya kapali olacak.
- Hoca sadece kendi olusturdugu testleri ve sorulari yonetecek.
- Hoca sadece kendisine tanimli ogrencileri ve kendi testlerine ait sonuc verilerini gorecek.
- Admin tum hocalari, dersleri ve eslestirmeleri yonetebilecek.
- Yetki kontrolu server-side fonksiyonlarda merkezi helper'lar uzerinden yapilacak.
- Client tarafindan gelen `teacher_id` , `owner_teacher_id` gibi alanlara guvenilmeyecek; aktif session kullanicisindan turetilecek.
- `NEXT_PUBLIC_` ile baslayan ortam degiskenleri sadece public degerler icin kullanılacak.
- `DATABASE_URL` , auth secret ve diger gizli anahtarlar `NEXT_PUBLIC_` prefix'i almadan Vercel environment variable olarak saklanacak.

6.6 UI ve Tasarim Sistemi

UI kararlar:

- Tailwind CSS ile responsive tasarim yapilacak.
- shadcn/ui ile button, card, input, dialog, dropdown, badge, table, sidebar ve form bilesenleri kullanılacak.
- Hoca panelinde sidebar + topbar yapisi olacak.
- Public ogrenci ekranlari sade, hizli ve mobil uyumlu olacak.
- Test ekraninda dikkat dagitmayacak temiz bir tasarim tercih edilecek.

UI durum yonetimi:

- Her veri cekme ekraninda uc durum tasarlanacaktir: yukleniyor (skeleton/spinner), bos durum (icerik yok mesaji ve yonlendirme) ve hata durumu (kullanici dostu Turkce mesaj ve tekrar dene butonu).
- Yukleniyor durumunda sayfa layout'u korunacak, skeleton componentleri kullanılacaktir.
- Bos durum ekranlarinda kullaniciya ne yapabilecegini anlatan bir mesaj ve aksiyon butonu gosterilecektir.
- Hata durumunda teknik detay gosterilmeyecek; "Bir sorun olustu, lutfen tekrar deneyin." mesaji ve tekrar dene butonu olacaktir.

6.7 Form ve Veri Dogrulama

Formlarda:

- React Hook Form kullanılacak.
- Zod ile schema bazli validasyon yapilacak.

- Öğrenci bilgi formu, soru ekleme formu, test oluşturma formu ve giriş formları ortak validasyon mantığıyla yazılacak.
- Hata mesajları kullanıcıya Türkçe ve net gösterilecek.

6.8 Test Stratejisi

MVP için test yaklaşımı:

- Kritik hesaplama fonksiyonları için unit test yazılacak.
- Test sonuç hesaplama, boş/dogru/yanlış sayımı ve puanlama mantığı ayrıca test edilecek.
- Öğrenci test akışı ve hoca girişi manuel senaryo testleriyle kontrol edilecek.
- İleri fazda Playwright ile uçtan uca testler eklenebilir.

6.9 Deployment Yaklaşımı

Deployment Vercel üzerinden yapılacak.

Ortamlar:

- Development: Lokal geliştirme
- Preview: Pull request veya branch bazlı Vercel preview
- Production: Canlı yayın

Environment variable kuralları:

- `DATABASE_URL` : Sadece server-side veya migration ihtiyacında kullanılır.
- `AUTH_SECRET` : Auth session imzalama için kullanılır, gizli tutulur.
- `AUTH_URL` : Deployment URL'sine göre ayarlanır.
- `GOOGLE_CLIENT_ID` : Google login eklenirse kullanılır.
- `GOOGLE_CLIENT_SECRET` : Google login eklenirse server-side gizli tutulur.
- `BLOB_READ_WRITE_TOKEN` : İleri fazda Vercel Blob ile dosya/görsel yükleme için kullanılır.

Gizli anahtarlar Git'e commit edilmeyecek. Vercel dashboard veya Vercel CLI üzerinden ortamlara ayrı ayrı tanımlanacak.

6.10 Yedekleme ve Geri Yükleme

Yedekleme stratejisi:

- Neon otomatik olarak point-in-time recovery desteği sunar; bu özellik aktif tutulacaktır.
- Production veritabanının haftalık manuel yedeği Neon dashboard üzerinden kontrol edilecektir.
- Kritik migration işlemlerinden önce veritabanının snapshot'i alınacaktır.
- Geri yükleme senaryosu en az bir kez test edilecektir.

6.11 Ozellik Karari

Online sinav sistemlerinde yaygin gorulen ozelliklerden hangilerinin MVP'de olacagi, hangilerinin ileri faza birakilacagi asagida netlestirilmistir. Sayfa ve akis bazinda MVP kapsamı için Bolum 12'ye bakilmalıdır.

MVP'de uygulanacak ozellikler:

- Soru havuzu (hoca bazlı)
- Ders bazlı test
- Zorluk seviyesi alanı
- Sureli veya suresiz test yapısı (test bazında seçilebilir)
- Otomatik sonuc hesaplama (server-side)
- Hoca panelinde sonuc listesi
- Öğrenci cevap detayları (sonuc ekranında gösterilir)
- Statik reklam alanları

İleri faza bırakılacak ozellikler:

- Sık karıştırma
- Soru sırası karıştırma
- Cevaplar için auto-save
- PDF sonuc raporu
- Gelişmiş analitik grafikler
- Çoklu soru tipi (çok cevaplı, doğru/yanlış, eşleme, vb.)
- Ortak soru havuzu
- Reklam ağı entegrasyonu (AdSense vb.)
- Sertifika/madalya verme

7. Netlestirilmis Veritabani Modelleri

Veritabanı PostgreSQL üzerinde kurulacaktır ve Prisma ORM ile yönetilecektir. Auth.js Prisma adaptörü `User`, `Account`, `Session` ve `VerificationToken` tablolarını otomatik oluşturur. Uygulamaya özel hoca/admin detayları ayrıca `profiles` tablosunda tutulacaktır.

7.1 profiles

Hoca ve admin kullanıcılarının uygulama profil bilgilerini tutar.

Alanlar:

- id
- user_id
- full_name
- email
- role: `admin` veya `teacher`

- avatar_url
- is_active
- created_at
- updated_at

Notlar:

- Hoca girişi Auth.js (NextAuth) ile yapılır.
- `user_id` , auth sistemindeki kullanıcı kaydına bağlanır.
- Yetki kontrolünde `profiles.role` ve server-side ownership kontrolleri kullanılır.
- Admin kullanıcılar ilk etapta manuel olarak tanımlanabilir.

7.2 students

Teste giren öğrencilerin bilgilerini tutar.

Alanlar:

- id
- full_name
- email
- phone
- grade_level
- school_name
- created_at
- updated_at

Notlar:

- Öğrenci ilk etapta üyelik acmadan test çözer.
- Aynı öğrencinin tekrar test çözebilmesi için telefon veya e-posta ile eşleştirme yapılabilir.
- İleri fazda öğrenci üyeliği eklenirse bu tablo auth kullanıcısıyla ilişkilendirilebilir.
- `email` alanı üzerinde `UNIQUE` constraint uygulanacaktır. Aynı e-posta ile birden fazla kayıt oluşturulamaz.
- `phone` alanı üzerinde `UNIQUE` constraint uygulanacaktır. Aynı telefon ile birden fazla kayıt oluşturulamaz.
- Yeni test girişinde mevcut öğrenci kaydı varsa güncellenir, yoksa yeni kayıt oluşturulur (upsert).

7.3 teacher_students

Hangi öğrencinin hangi hocaya tanımlı olduğunu tutar.

Alanlar:

- id
- teacher_id
- student_id
- assigned_by

- assigned_at
- note

Notlar:

- `teacher_id` , `profiles.id` alanina baglanir.
- `student_id` , `students.id` alanina baglanir.
- Admin, hoca-ogrenci eslestirmesini bu tablo uzerinden yapar.
- Bu yapi, ileride bir ogrencinin birden fazla hocaya baglanmasina da izin verir.

7.4 courses

Dersleri tutar.

Alanlar:

- id
- title
- slug
- description
- icon_name
- display_order
- is_active
- created_at
- updated_at

Notlar:

- Dersler public tarafta listelenir.
- Pasif dersler ogrenci tarafinda gosterilmez.

7.5 questions

Soru havuzunu tutar.

Alanlar:

- id
- course_id
- owner_teacher_id
- question_text
- option_a
- option_b
- option_c
- option_d
- correct_option

- difficulty: `easy` , `medium` , `hard`
- topic
- explanation
- image_url
- is_active
- created_at
- updated_at

Notlar:

- MVP'de soru tipi coktan secmeli ve tek dogru cevapli olacak.
- `topic`alani konu bazli filtreleme icin kullanilir.
- `explanation`alani sonuc detayinda veya ilerleyen fazda cevap aciklamasi icin kullanilir.
- Ortak soru havuzu istenirse ilerleyen fazda `visibility` veya `is_shared`alani eklenebilir.

7.6 tests

Hocalarin olusturdugu testleri tutar.

Alanlar:

- id
- course_id
- owner_teacher_id
- title
- description
- duration_minutes
- status: `draft` , `active` , `archived`
- show_result_immediately
- allow_anonymous_attempts
- starts_at
- ends_at
- created_at
- updated_at

Notlar:

- Ogrenci tarafinda sadece `active`durumdaki testler gorunur.
- `allow_anonymous_attempts`ilk MVP'de true olarak kullanilabilir.
- Sure zorunlu olmayabilir; `duration_minutes`bos ise suresiz test anlamina gelir.
- `starts_at`ve `ends_at`alanlari dolu olan testlerde status gecisi otomatik olarak kontrol edilecektir: `starts_at`gecmis ve `ends_at`gelecekteyse test `active`sayilir, `ends_at`gecmisse `archived`sayilir.
- Bu kontrol, testin sorgulandigi anda server-side yapilacaktir (cron job yerine lazy evaluation).

7.7 test_questions

Test ve soru iliskisini tutar.

Alanlar:

- id
- test_id
- question_id
- order_index
- points (varsayilan: 1)
- created_at

Notlar:

- MVP'de her soru 1 puan olarak hesaplanir.
- Ileri fazda farkli puan vermek icin `points`alani teste gore degistirilebilir.

7.8 test_attempts

Ogrencinin test girisini tutar.

Alanlar:

- id
- test_id
- student_id
- status: `in_progress`, `completed`, `abandoned`
- started_at
- completed_at
- duration_seconds
- score
- correct_count
- wrong_count
- empty_count
- ip_address
- user_agent
- created_at

Notlar:

- Ogrenci testi baslattiginda `in_progress` kaydi acilir.
- Test bitince skor ve sayim alanlari doldurulur.
- Ileri fazda yarim kalan testleri tamamlama veya auto-save bu tablo uzerinden gelistirilebilir.

7.9 student_answers

Ogrenci cevaplarini tutar.

Alanlar:

- id
- attempt_id
- question_id
- selected_option (nullable)
- is_correct
- answered_at

Notlar:

- Bos birakilan sorularda `selected_option` `null` olarak kaydedilir; `is_correct` `false` olur.
- Sonuc detay sayfasi bu tabloyu kullanir.
- Test bittiginde tum cevaplar tek bir veritabani transaction'i icinde batch insert ile kaydedilir. Bu hem performansi arttirir hem de yarim kalan kayitlari onler.

7.10 ads

Reklam bolumlerini tutar. Ilk fazda statik reklam alanlari kullanilabilir; dinamik reklam yonetimi ilerleyen fazda eklenir.

Alanlar:

- id
- title
- description
- image_url
- target_url
- placement: `home` , `course_list` , `result`
- is_active
- starts_at
- ends_at
- created_at
- updated_at

7.11 Sistem Ayarlari

Ilari faz icin `settings` tablosu eklenebilir.

Kullanilabilecek alanlar:

- id
- key

- value
- updated_by
- updated_at

Kullanım alanlari:

- Site adi
- Varsayılan test suresi
- Reklam ayarlari
- Bakim modu

7.12 Yetkilendirme ve Veri Erisimi Taslagi

Veritabani seviyesinde RLS kullanilmadigi icin tum veri erisimi Next.js server action, route handler ve servis fonksiyonlari icinde kontrol edilir. Client componentler dogrudan veritabani sorgusu calistirmaz.

Veri erisim kurallari:

- `profiles` : Kullanici kendi profilini gorebilir. Admin tum profilleri yonetebilir.
- `students` : Admin tum ogrencileri gorebilir. Hoca sadece `teacher_students` uzerinden kendisine bagli ogrencileri gorebilir.
- `teacher_students` : Admin tum eslestirmeleri yonetebilir. Hoca kendi eslestirmelerini okuyabilir.
- `questions` : Hoca kendi sorularini yonetebilir. Admin tum sorulari gorebilir.
- `tests` : Hoca kendi testlerini yonetebilir. Public tarafta sadece aktif testler okunabilir.
- `test_attempts` : Hoca kendi testlerine ait denemeleri gorebilir. Admin tum denemeleri gorebilir.
- `student_answers` : Hoca kendi testlerine ait cevaplari gorebilir. Admin tum cevaplari gorebilir.
- `ads` : Public tarafta aktif reklamlar okunabilir. Admin reklam yonetimi yapabilir.

Teknik uygulama:

- `requireTeacher()` aktif session kullanicisinin hoca oldugunu kontrol eder.
- `requireAdmin()` aktif session kullanicisinin admin oldugunu kontrol eder.
- `getTeacherProfile()` session kullanicisindan hoca profilini getirir.
- `assertOwnsQuestion(questionId)` sorunun aktif hocaya ait oldugunu dogrular.
- `assertOwnsTest(testId)` testin aktif hocaya ait oldugunu dogrular.
- `assertCanViewAttempt(attemptId)` hocanin ilgili test sonucunu gorebilecegini dogrular.

7.13 MVP Disi Ama Planlanan Veri Ozellikleri

Ilere fazlarda eklenebilecek veri ozellikleri:

- Coklu soru tipi
- Sik karistirma
- Soru sirasi karistirma
- PDF sonuc raporu

- Öğrenci üyeliği
- Reklam tıklanma istatistikleri
- Hoca bazlı detaylı analizler
- Değişiklik izleme (audit trail)

7.14 Değişiklik İzleme (İleri Faz)

İleri fazda `audit_logs` tablosu eklenecektir:

- id
- user_id
- action: `create`, `update`, `delete`
- entity_type: `question`, `test`, `student` vb.
- entity_id
- old_values (JSON)
- new_values (JSON)
- created_at

Bu tablo sayesinde hoca bir soruyu değiştirdiğinde eski hali kaybolmaz ve admin tarafından denetlenebilir.

8. Fazlara Bolunmuş Yol Haritası

Faz 0: Proje Hazırlığı

Durum: Büyük ölçüde tamamlandı. PRD ve mimari kararlar bu dokümanda yer alır.

Amac: Projenin temel kurallarını, teknoloji seçimini ve iş bölümünü netleştirmek.

Tamamlanan işler:

- PRD ilk sürümü oluşturuldu.
- Teknoloji stack'i seçildi (bkz. Bölüm 6).
- Veritabanı modeli taslağı çıkarıldı (bkz. Bölüm 7).
- İş bölümü netleştirildi (bkz. Bölüm 9 ve 10).

Bu fazda kalan işler:

- Git repo kurulumu ve branch düzeni.
- Tasarım dili için ortak referans (renk paleti, font, ikon seti).
- Rakip/benzer site referans incelemesi.

Sorumluluk:

- Arda: Repo kurulumu, branch ve commit kuralları.
- Emir: Tasarım dili, renk/font referansı, rakip site incelemesi.

Kabul kriterleri:

- Repo hazir ve iki kisi de erisebiliyor.
- Tasarim dili icin ortak bir referans belirlenmis.
- Faz 1'e baslamak icin tum kararlar netlesmis.

Faz 1: Proje Iskeleti ve Ana Sayfa

Amac: Uygulamanin temel iskeletini ve ilk gorunen yuzunu olusturmak.

Isler:

- Proje kurulumu.
- Sayfa routing yapisi.
- Ortak layout.
- Header ve footer.
- Ana sayfa.
- Hoca giris butonu.
- Online test butonu.
- Statik reklam alani.

Sorumluluk:

- Arda: Proje kurulumu, routing, layout yapisi.
- Emir: Ana sayfa tasarimi, header/footer, reklam alani.

Kabul kriterleri:

- Ana sayfa acilir.
- Online test butonu ders secim sayfasina gider.
- Hoca giris butonu giris sayfasina gider.
- Reklam alani gorunur.

Faz 2: Dersler ve Ogrenci Test Akisi

Amac: Ogrencinin ders secip teste baslayabildigi temel akisi kurmak.

Isler:

- Ders modeli.
- Ders listeleme sayfasi.
- Test baslangic sayfasi.
- Ogrenci bilgi formu.
- Test oturumu baslatma.

Sorumluluk:

- Arda: Ders verisi, test oturumu mantigi, form validasyonu.
- Emir: Ders kartlari, ogrenci bilgi formu UI, bos durum tasarımlari.

Kabul kriterleri:

- Öğrenci dersleri görebilir.
- Bir derse basınca test başlangıç sayfasına gider.
- Öğrenci bilgilerini girerek teste başlayabilir.

Faz 3: Test Çözme ve Sonuç Hesaplama

Amac: Öğrencinin testi tamamlayıp sonucunu görebilmesini sağlamak.

İşler:

- Soru gösterme ekranı.
- Sık seçme.
- Cevapları geçici olarak tutma.
- Testi bitirme.
- Doğru/yanlış/bos hesaplama.
- Sonuç ekranı.
- Sonucu veritabanına kaydetme.

Sorumluluk:

- Arda: Test motoru, cevap hesaplama, veritabanı kaydı.
- Emir: Soru kartı UI, sonuç ekranı tasarımı.

Kabul kriterleri:

- Öğrenci soruları cevaplayabilir.
- Testi bitirebilir.
- Sonucunu görebilir.
- Sonuç sistemde kayıtlı kalır.

Faz 4: Hoca Girişi ve Panel

Amac: Hocaların güvenli şekilde panele girmesini sağlamak.

İşler:

- Hoca giriş sayfası.
- Auth.js entegrasyonu ve session yönetimi.
- Rol kontrolü (admin ve teacher).
- Korumalı route ve middleware.
- Hoca panel layout'u.
- Panel ana sayfası.
- Admin tarafından ilk hoca hesaplarının manuel olarak oluşturulması.

Sorumluluk:

- Arda: Auth, rol kontrolu, korumali sayfalar.
- Emir: Giris formu UI, panel layout gorsel duzen.

Kabul kriterleri:

- Hoca giris yapabilir.
- Giris yapmayan kisi hoca paneline giremez.
- Hoca panel ana sayfasini gorur.

Faz 5: Soru Havuzu

Amac: Hocalarin soru yukleyebilmesini ve havuzdan soru gorebilmesini saglamak.

Isler:

- Soru ekleme formu.
- Soru listeleme.
- Ders filtresi.
- Zorluk filtresi.
- Soru duzenleme.
- Soru pasife alma.

Sorumluluk:

- Arda: Soru CRUD, yetki kontrolu, veritabani islemleri.
- Emir: Soru liste UI, form alanlari, filtre arayuzu.

Kabul kriterleri:

- Hoca soru ekleyebilir.
- Soru havuzunda sorular listelenir.
- Hoca sorulari derse gore filtreleyebilir.
- Hoca soru duzenleyebilir veya pasife alabilir.

Faz 6: Test Olusturma

Amac: Hocalarin soru havuzundan test olusturabilmesini saglamak.

Isler:

- Test olusturma formu.
- Ders secimi.
- Havuzdan soru secimi.
- Test soru sirasi.
- Test aktif/pasif ayari.

Sorumluluk:

- Arda: Test olusturma mantigi, test_questions iliskisi, aktif test sorgulari.

- Emir: Test formu UI, soru secim listesinin gorsel duzeni.

Kabul kriterleri:

- Hoca yeni test olusturabilir.
- Teste soru ekleyebilir.
- Testi aktif yapabilir.
- Aktif test ogrenci tarafinda gorunur.

Faz 7: Ogrenci ve Sonuc Takibi

Amac: Hocalarin sinava giren ogrencileri ve sonuclari takip etmesini saglamak.

Isler:

- Sinava giren ogrenciler listesi.
- Ogrenci detay sayfasi.
- Test sonucu detaylari.
- Kendisine tanimli ogrenciler listesi.
- Admin tarafından hoca-ogrenci eslestirme mantigi.

Sorumluluk:

- Arda: Sorgular, yetki kontrolu, hoca-ogrenci eslestirme, detay sayfalari.
- Emir: Liste ve tablo tasarimlari, bos durumlar ve filtre alanlari.

Kabul kriterleri:

- Hoca sinava giren ogrencileri gorebilir.
- Hoca ogrenci sonuc detayini gorebilir.
- Hoca kendisine tanimli ogrencileri gorebilir.
- Hoca yetkisi olmayan veriye erisemez.

Faz 8: Reklam Alanlari

Amac: Reklam bolumlerini kullanilabilir hale getirmek.

Isler:

- Ana sayfa reklam alani.
- Ders sayfasi reklam alani.
- Sonuc sayfasi reklam alani.
- Ileri fazda admin panelden reklam yonetimi.

Sorumluluk:

- Emir: Statik reklam alanlari, gorsel duzen, placeholder tasarimlar.
- Arda: Dinamik reklam modeli ve admin yonetimi, ileri faz.

Kabul kriterleri:

- Reklam alanlari sayfalarda gorunur.
- Reklam alanlari tasarimi bozmaz.
- Ileri faz icin veri modeli hazirdir.

Faz 9: Test, Temizlik ve Yayina Hazirlik

Amac: Projeyi kullanilabilir ve sunulabilir hale getirmek.

Isler:

- Kritik akislari test etmek.
- Mobil uyumluluk kontrolu.
- Form validasyonlarini kontrol etmek.
- Hata mesajlarini duzenlemek.
- Seed/demo verileri hazirlamak.
- Deployment.

Sorumluluk:

- Arda: Teknik testler, deployment, veri guvenligi, performans kontrolu.
- Emir: Mobil tasarim kontrolu, metin duzeltmeleri, demo veri girisi, UI hatalarini raporlama.

Kabul kriterleri:

- Ogrenci testi bastan sona cozebilir.
- Hoca soru ve test yonetebilir.
- Hoca sonuclari gorebilir.
- Site mobilde kullanilabilir.
- Demo yayin linki hazirdir.

9. Arda Icin Net Gorev Sahipligi

Arda asagidaki modullerin tamamlanmasindan sorumludur:

9.1 Proje Altyapisi

- Proje kurulumunu yapmak.
- Klasor yapisini belirlemek.
- Routing yapisini kurmak.
- Ortak layout mantigini hazirlamak.
- Ortam degiskenleri ve proje ayarlarini duzenlemek.

9.2 Veritabani ve Veri Modeli

- Veritabani tablolarini tasarlamak.

- `profiles` , `students` , `teacher_students` , `courses` , `questions` , `tests` , `test_questions` , `test_attempts` , `student_answers` tablolarini olusturmak.
- Tablo iliskilerini ve gerekli indeksleri planlamak.
- Demo veri veya seed yapisini hazirlamak.

9.3 Auth ve Yetkilendirme

- Hoca giris sistemini kurmak.
- Admin ve teacher rollerini ayirmak.
- Hoca panelini giris yapmayan kullanicilara kapatmak.
- Hocalarin yalnızca yetkili olduklari verilere erismesini saglamak.

9.4 Ogrenci Test Akisi

- Ogrenci bilgi formundan gelen veriyi dogrulamak.
- Test oturumu baslatma mantigini yazmak.
- Sorularin ogrenciye dogru sekilde gelmesini saglamak.
- Cevaplari kaydetmek.
- Test bitince sonucu hesaplamak.
- Sonucu veritabanina kaydetmek.

9.5 Soru Havuzu ve Test Olusturma

- Soru ekleme, duzenleme, listeleme ve pasife alma islemlerini yazmak.
- Soru havuzunu ders ve zorluk bilgisine gore sorgulanabilir hale getirmek.
- Test olusturma mantigini yazmak.
- Test ile soru iliskisini kurmak.
- Aktif/pasif test durumunu yönetmek.

9.6 Hoca Paneli Veri Akislari

- Dashboard icin gerekli sayisal verileri hazirlamak.
- Sinava giren ogrenciler listesini olusturmak.
- Ogrenci sonuc detaylarini getirmek.
- Kendisine tanimli ogrenciler akisini kurmak.
- Hoca-ogrenci eslestirme mantigini hazirlamak.

9.7 Admin Paneli Veri Akislari

- Admin paneli icin korumali rotalari hazirlamak.
- Hoca hesabi olusturma, duzenleme ve pasife alma akisini yazmak.
- Ders olusturma ve duzenleme akisini yazmak.
- Hoca-ogrenci eslestirme yonetimini yazmak.

- Reklam yonetimi backend mantigini ileri fazda hazirlamak.

9.8 Yayina Alma ve Kontrol

- Kritik akislari test etmek.
- Deployment ayarlarini yapmak.
- Ortam degiskenlerini yayina hazirlamak.
- Son teknik kontrol ve kod review yapmak.

10. Emir Icin Net Gorev Sahipligi

Emir asagidaki modullerin tamamlanmasindan sorumludur:

10.1 Ana Sayfa ve Ortak Arayuz

- Ana sayfa tasarimini yapmak.
- Header componentini hazirlamak.
- Footer componentini hazirlamak.
- "Online Teste Basla" ve "Hoca Girisi" yonlendirme butonlarini arayuzde konumlandirmek.
- Ana sayfadaki tanitim metinlerini ve gorsel bolumleri hazirlamak.

10.2 Ders Secim Arayuzu

- Ders karti componentini hazirlamak.
- Ders listeleme sayfasinin gorsel duzenini yapmak.
- Aktif test olmayan dersler icin bos durum tasarimini hazirlamak.
- Ders kartlarinin mobil gorunumunu kontrol etmek.

10.3 Ogrenci Test Arayuzleri

- Test baslangic sayfasi arayuzunu hazirlamak.
- Ogrenci bilgi formu alanlarini tasarlamak.
- Test kurallari bolumunu hazirlamak.
- Soru karti gorsel duzenini yapmak.
- Siklarin secilebilir gorunumunu hazirlamak.
- Onceki, sonraki ve testi bitir butonlarinin arayuzunu hazirlamak.
- Sonuc ekrani tasarimini yapmak.

10.4 Hoca Paneli Arayuzleri

- Hoca giris formu arayuzunu hazirlamak.
- Hoca panel layout'unun gorsel duzenini yapmak.
- Dashboard kartlarini tasarlamak.

- Soru havuzu liste ekraninin arayuzunu hazirlamak.
- Soru ekleme/duzenleme formunun arayuzunu hazirlamak.
- Test olusturma formunun arayuzunu hazirlamak.
- Sinava giren ogrenciler liste/tablo gorunumunu hazirlamak.
- Kendisine tanimli ogrenciler liste gorunumunu hazirlamak.

10.5 Admin Paneli Arayuzleri

- Admin panel layout'unu hazirlamak.
- Hoca yonetimi liste ve form arayuzunu hazirlamak.
- Ders yonetimi liste ve form arayuzunu hazirlamak.
- Hoca-ogrenci eslestirme arayuzunu hazirlamak.
- Reklam yonetimi arayuzunu ileri fazda hazirlamak.

10.6 Reklam ve Icerik Alanlari

- Ana sayfa reklam alanini hazirlamak.
- Ders secim sayfasi reklam alanini hazirlamak.
- Sonuc sayfasi reklam alanini hazirlamak.
- Reklam alanlari icin placeholder metin ve gorselleri hazirlamak.
- Sayfa metinlerini ve demo icerikleri duzenlemek.

10.7 Responsive ve Gorsel Kontrol

- Ana sayfa mobil gorunumunu kontrol etmek.
- Ders secim ve test ekranlarinin mobil gorunumunu kontrol etmek.
- Hoca panelindeki liste ve kartlarin responsive davranisini kontrol etmek.
- Bos durum, hata durumu ve yukleniyor durumlari icin arayuz hazirlamak.

10.8 Ortak Teslim Kurallari

- Her gorevin sonunda ilgili ekran calisir halde teslim edilir.
- Arda, Emir'in hazirladigi arayuzleri veri ve is mantigina baglar.
- Emir, Arda'nin hazirladigi veri akislari icin gerekli arayuz duzenlemelerini tamamlar.
- Faz sonunda iki kisi birlikte ogrenci ve hoca akislarini kontrol eder.

11. Onerilen Haftalik Calisma Plani

Haftalik plan 6 haftaya yayilmistir ve Bolum 8'deki fazlarla esleserek ilerler.

Hafta 1: Faz 0 ve Faz 1 (Hazirlik + Iskelet)

Hedef:

- Repo kurulumu, branch ve commit duzeni
- Tasarim dili referansi
- Next.js + TypeScript + Tailwind + shadcn/ui kurulumu
- Ortak layout, header, footer
- Ana sayfa ilk hali

Arda:

- Repo kurulumu, Next.js + Prisma + Auth.js iskeleti.
- Route gruplari ve layout yapisi.
- `.env` ve Vercel ortam degisken duzeni.

Emir:

- Tasarim dili referansi (renk, font).
- Ana sayfa tasarimi.
- Header ve footer componenti.
- Statik reklam alanı placeholder'i.

Hafta 2: Faz 2 (Dersler ve Ogrenci Akisi)

Hedef:

- Ders modeli ve listeleme sayfası
- Test baslangic sayfası
- Ogrenci bilgi formu

Arda:

- Prisma migration ile `courses`, `students`, `tests`, `test_questions` tablolari.
- Test oturumu baslatma server action.
- Form validasyonu (Zod).

Emir:

- Ders kartlari ve liste sayfası UI.
- Ogrenci bilgi formu UI.
- Bos durum ekranlari.

Hafta 3: Faz 3 (Test Cozme ve Sonuc)

Hedef:

- Test cozme ekrani
- Cevap kaydi ve sonuc hesaplama
- Sonuc ekrani

Arda:

- `test_attempts` ve `student_answers` migration ve servis fonksiyonlari.
- Sonuc hesaplama (dogru/yanlis/bos/puan).
- Server-side sure kontrolu.

Emir:

- Soru karti UI.
- Sik secimi ve gezinme.
- Sonuc ekrani tasarimi.

Hafta 4: Faz 4 ve Faz 5 (Hoca Girisi + Soru Havuzu)

Hedef:

- Auth.js entegrasyonu
- Hoca panel layout'u
- Soru havuzu CRUD

Arda:

- Auth.js Prisma adaptoru ve session yonetimi.
- Korumali rota ve middleware.
- `questions` CRUD ve yetki kontrolu.

Emir:

- Giris formu UI.
- Hoca panel layout'u (sidebar + topbar).
- Dashboard kartlari.
- Soru havuzu liste ve form UI.

Hafta 5: Faz 6 ve Faz 7 (Test Olusturma + Sonuc Takibi)

Hedef:

- Test olusturma akisi
- Sinava giren ogrenciler
- Hoca-ogrenci eslestirme
- Admin paneli ilk hali

Arda:

- Test olusturma servisi ve `test_questions` iliskisi.
- Hoca-ogrenci eslestirme ve admin servisleri.
- Sinava giren ogrenciler sorgulari.

Emir:

- Test olusturma formu UI.
- Sinava giren ogrenciler tablosu.
- Tanimli ogrenciler liste UI.
- Admin paneli liste arayuzleri.

Hafta 6: Faz 8 ve Faz 9 (Reklam + Yayina Hazirlik)

Hedef:

- Reklam alanlari
- KVKK ve icerik sayfalari
- Test guvenligi gozden gecirme
- Mobil ve gorsel son kontroller
- Deployment

Arda:

- Test guvenligi kontrolleri (yeniden giris, sure, server-side dogrulama).
- KVKK ve icerik sayfalari icin route hazirlama.
- Vercel production deployment.
- Hata yonetimi ve loglama gozden gecirme.

Emir:

- Reklam alanlarinin tamamlanmasi.
- Hakkimizda, Iletisim, SSS, Gizlilik, KVKK metni, Kullanim Kosullari sayfalari.
- Mobil gorunum kontrolleri.
- Erisilebilirlik ve icerik son okumasi.

12. MVP Kapsami (Sayfa ve Akis Bazinda)

Bu bolum, MVP'de yer alacak sayfalari ve akislari listeler. Ozellik bazinda kapsam icin Bolum 6.10'a bakilmalidir.

Ilk calisan surumde olacak sayfalar ve akislar:

- Ana sayfa
- "Online Test"e yonlendirme
- Ders listeleme sayfasi
- Test baslangic sayfasi (ogrenci bilgi formu + test kurallari + KVKK onay)
- Test cozme ekrani
- Sonuc ekrani (ozet + cevap detayi)
- Hoca giris sayfasi
- Hoca paneli ana sayfasi
- Soru havuzu sayfasi (liste + ekle/duzenle)
- Test yonetimi sayfasi (liste + ekle/duzenle)

- Sinava giren ogrenciler sayfasi
- Tanimli ogrenciler sayfasi
- Admin paneli (hoca ve ders yonetimi)
- Icerik sayfalari (Hakkimizda, Iletisim, SSS, Gizlilik, KVKK, Kullanim Kosullari)
- Statik reklam alanlari (ana sayfa, ders listesi, sonuc)

Ilk surumde olmayacak sayfalar ve akislar:

- Ogrenci uyelig ve oturum yonetimi
- Odeme sayfasi
- Detayli istatistik/grafik sayfalari
- Sertifika sayfasi
- Coklu dil destegi
- SMS/e-posta bildirim ayarlari
- Yapay zeka ile soru uretimi
- Reklam yonetim paneli (admin tarafinda dinamik reklam yonetimi)

13. Verilmis Kararlar

PRD'nin tek karar kaynagi olmasi prensibiyle, projenin ilerleyisini etkileyen kararlar bu bolumde net olarak yazilmistir. Bir karar degistirilmek istendiginde once bu bolum guncellenir, sonra kod yazilir.

13.1 Ogrenci Bilgi Toplama

- Ogrenci, teste baslamadan once asagidaki bilgileri girer:
 - Ad-soyad (zorunlu)
 - Telefon **veya** e-posta (en az biri zorunlu)
 - Sinif seviyesi (zorunlu)
 - Bagli oldugu hoca (opsiyonel, dropdown)
- Ogrenci girisi/uyelig MVP'de yoktur. Tekrar test cozme kontrolu telefon/e-posta uzerinden yapilir.

13.2 Hoca Hesabi Yonetimi

- Hocalar kendileri kayit olamaz.
- Tum hoca hesaplari admin tarafindan olusturulur.
- Public bir kayit sayfasi MVP'de yoktur.

13.3 Aktif Test Sayisi

- Bir ders icinde ayni anda birden fazla aktif test olabilir.
- Ogrenci bir dersi sectiginde aktif testlerin listesini gorur.

13.4 Soru Havuzu Sahipligi

- Sorular hoca bazlidir; her hoca yalnızca kendi sorularini gorur ve teste ekler.
- Ortak soru havuzu MVP'de yoktur, ileri faza birakilmistir.

13.5 Reklam Modeli

- Reklamlar MVP'de statiktir.
- Admin panelinden dinamik reklam yonetimi ileri faza birakilmistir.
- Reklam ag entegrasyonu (AdSense vb.) MVP'de yoktur.

13.6 Soru Silme

- Sorular kalici olarak silinmez.
- `is_active = false` ile pasife alinir (soft-delete).
- Bir teste ekli sorular pasife alinsa bile o testlerde gorunmeye devam eder.

13.7 Tekrar Deneme

- Ayni testin ayni ogrenci tarafından tekrar tekrar cozulmesi MVP'de engellenir.
- Tekil deneme kontrolu `test_attempts` tablosunda `(test_id, student_id)` uzerinden unique index ile saglanir.
- Ogrenci kimligi OTP ile dogrulanmis telefon/e-posta uzerinden belirlenir (bkz. Bolum 4.1 Kimlik dogrulama).
- OTP dogrulasindan gecmis ogrenci kaydi `students` tablosundaki mevcut kayitle eslestirilir.
- "Tekrar denemeye izin ver" ayari ileri faza birakilmistir.

13.8 Sure Yonetimi

- Test bazinda `duration_minutes` belirlenebilir.
- Bos birakilirsa test suresizdir.
- Sure server-side kontrol edilir ve dolunca cevap kabul edilmez.

13.9 Dil

- Tek dil: Turkce.
- Coklu dil destegi MVP'de yoktur.

13.10 Yasal

- Aydinlatma metni ve KVKK onay kutusu test baslamadan once gosterilir.
- Gizlilik politikasi, KVKK aydinlatma metni ve kullanim kosullari sayfari yayina alinmadan once hazir olur.

13.11 Esanli Duzenleme

- Iki hocanın ayni anda ayni soruyu veya testi duzenlemesi MVP'de engellenmez.

- Son kaydi yapan kisinin degisiklikleri gecerli olur (last-write-wins).
- Ileri fazda `updated_at`alani uzerinden optimistic concurrency control ("Bu kayit baskasi tarafindan degistirildi" uyarisi) eklenebilir.

13.12 E-posta Servisi

- OTP gondermek, sifre sifirlama ve bildirimler icin Resend kullanılacaktır.
- Resend, Vercel ile uyumlu ve ucretsiz katmanda gunluk 100 e-posta destegi sunar.
- `RESEND_API_KEY` environment variable olarak saklanacaktır.
- E-posta sablonlari React Email ile olusturulacaktır.
- SMS entegrasyonu (Twilio, Netgsm vb.) ileri faza birakilmistir.

14. Basari Kriterleri

Proje basarili sayilacaksa:

- Ogrenci uye olmadan test cozebilmeli.
- Test sonucu dogru hesaplanmali ve kaydedilmeli.
- Hoca giris yapip panelini gorebilmeli.
- Hoca soru ekleyip test olusturabilmeli.
- Hoca sinava giren ogrencileri gorebilmeli.
- Hoca kendisine tanimli ogrencileri gorebilmeli.
- Sistem mobilde temel olarak kullanilabilir olmalidir.
- Reklam alanlari sayfalarda yer almalidir.

15. Dosya ve Kod Organizasyonu Kurallari

Proje kodlanirken:

- Her dosya mumkunse 500 satiri gecmemeli.
- Sayfalar, componentler, servisler ve veritabanı islemleri ayrilmali.
- Tekrarlanan UI parcalari component haline getirilmeli.
- Is mantigi componentlerin icine dagitilmamali.
- Kritik kararlar once bu PRD dosyasina eklenmeli.
- Faz disisler ayri not edilmeli ve plan disina cikilmamali.

Onerilen klasor mantigi:

- `src/app` : Next.js App Router sayfaları ve route grupları
- `src/app/(public)` : Ana sayfa ve public ders/test giris ekranları
- `src/app/(student)` : Ogrenci test akisi
- `src/app/(teacher)` : Hoca paneli
- `src/app/(admin)` : Admin paneli

- `src/components` : Ortak UI bileşenleri
- `src/components/ui` : shadcn/ui bileşenleri
- `src/features` : Öğrenci test akisi, hoca paneli, soru havuzu gibi modüller
- `src/lib` : Prisma client, auth yardımcıları, validasyon ve servis fonksiyonları
- `src/types` : Ortak TypeScript tipleri
- `src/styles` : Global stiller
- `prisma` : Prisma schema, migration ve seed dosyaları

Onerilen feature klasörleri:

- `src/features/student-test`
- `src/features/teacher-dashboard`
- `src/features/question-bank`
- `src/features/test-builder`
- `src/features/results`
- `src/features/ads`

16. İlk Yapılacaklar

1. Next.js + TypeScript + Tailwind CSS projesini kur.
2. shadcn/ui kurulumunu yap ve temel UI componentlerini ekle.
3. Prisma kurulumunu yap.
4. PostgreSQL için Neon üzerinde veritabanı oluştur.
5. Auth.js (NextAuth) Prisma adaptoru kurulumunu hazırla.
6. `profiles` , `students` , `teacher_students` , `courses` , `questions` , `tests` , `test_questions` , `test_attempts` , `student_answers` tablolarını Prisma migration olarak hazırla.
7. Vercel deployment için environment variable listesini oluştur.
8. Ana sayfa ve route gruplarını oluştur.
9. Ders listeleme ve öğrenci test akisini başlat.
10. Her faz sonunda PRD'yi güncelleyip ilerlemeyi kontrol et.

17. Arastırma Kaynakları

Bu PRD güncellemesinde dikkate alınan kaynaklar:

- Online sınav sistemlerinde yaygın özellikler: soru havuzu, sürekli test, otomatik sonuç, öğrenci cevap detayları, hoca/admin dashboard'u ve raporlama.
- Prisma + Auth.js + Next.js dokümanı: <https://prisma.io/docs/guides/authjs-nextjs>
- Next.js App Router proje yapısı dokümanı: <https://nextjs.org/docs/app/getting-started/project-structure>
- shadcn/ui dokümanı: <https://ui.shadcn.com/docs>

- Vercel environment variables dokumani: <https://vercel.com/docs/environment-variables/manage-across-environments>
- Firebase Security Rules dokumani: <https://firebase.google.com/docs/rules/rules-and-auth>
- Appwrite Auth dokumani: <https://appwrite.io/docs/advanced/security/authentication>
- PocketBase production dokumani: <https://pocketbase.io/docs/going-to-production>
- Laravel Sanctum dokumani: <https://laravel.com/docs/11.x/sanctum>

Arastirmadan cikan ana sonuc:

- Bu proje icin secilen MVP yaklasimi; Next.js App Router ile tek uygulama, Prisma ile PostgreSQL veritabani erisimi, Auth.js (NextAuth) ile hoca/admin girisi, Tailwind + shadcn/ui ile arayuz ve Vercel ile deployment seklindedir.
- Ayri backend uygulaması ilk surum icin gerekli degildir.
- Yetki kontrolu sadece frontend kontrollerine birakilmayacak, server-side servis fonksiyonlari ve ownership kontrolleriyle uygulanacaktır.

18. Gizlilik, KVKK ve Hukuki Notlar

Bu bolum, Turkiye'de yayina cikacak bir online test platformu icin minimum gerekli yasal ve gizlilik gerekliliklerini ozetler.

18.1 Toplanacak Veriler

Sistem asagidaki kisisel verileri toplar:

- Ogrenci ad-soyad, e-posta, telefon, sinif/okul bilgisi
- Ogrenci test cevapları ve sonuc verileri
- `test_attempts` tablosunda `ip_address` ve `user_agent`
- Hoca ve admin hesap bilgileri (e-posta, ad-soyad)

18.2 KVKK Gereklilikleri

- Aydinlatma metni hazirlanacak ve "Teste Basla" sayfasinda kullaniciya gosterilecek.
- Ogrenci, kisisel verilerinin islenmesini onaylamadan teste baslayamayacak.
- Kullanici verilerinin silinme talepleri icin iletisim adresi yayınlanacak.
- Veriler sadece sistemin calismasi icin gerekli sure boyunca saklanacak.
- Saglik, dini inanc gibi özel nitelikli veri toplanmayacak.

18.3 Cerez ve KVKK Bildirimi

- Ana sayfada cerez bildirimi banner'i olacak.
- Sadece zorunlu cerezler kullanılacak; uclu parti analytics eklenirse onay alınacak.

18.4 Reklam Tarafi

- Reklamlar ilk fazda statik olacagi icin reklam ag'lari (Google AdSense vb.) MVP'de kullanilmayacak.
- Reklam ag'i eklenirse ek aydinlatma metni guncellemesi gerekecek.

19. Hata Yonetimi, Loglama ve Izleme

19.1 Hata Yonetimi

- Tum server action ve route handler'lar try/catch ile saridir.
- Kullaniciya teknik hata mesaji gosterilmez; "Bir sorun olustu, lutfen tekrar deneyin." gibi sade Turkce mesajlar gosterilir.
- Form validasyon hatalari React Hook Form + Zod uzerinden alan bazli gosterilir.
- Bilinmeyen route'lar icin `not-found.tsx` sayfasi olur.
- Beklenmeyen hatalar icin `error.tsx` sayfasi olur.

19.2 Loglama

- Gelistirme: Konsol loglari yeterli.
- Production: Vercel Logs uzerinden temel inceleme yapilir.
- Kritik hatalar icin Sentry entegrasyonu ileri fazda dusunulebilir.

19.3 Analytics

- Vercel Analytics ile sayfa görüntülemeleri ve performans izlenir.
- Reklam tıklamaları ileri fazda kendi tablomuza loglanabilir.

20. SEO ve Metadata

- Public sayfalarda Next.js metadata API kullanilarak `title`, `description` ve `og:image` belirlenir.
- Site genel `sitemap.xml` ve `robots.txt` dosyalari olusturulur.
- Public sayfa URL'leri Turkce ve okunabilir olacaktir (ornegin `/online-test`, `/dersler`).
- Hoca paneli ve admin sayfalari `noindex` olarak isaretlenir.

21. Erisilebilirlik ve Tarayici Destegi

- Hedef erisilebilirlik seviyesi: WCAG 2.1 AA (mumkun olan oranda).
- shadcn/ui Radix tabanli oldugu icin temel klavye ve ekran okuyucu destegi gelir.
- Renk kontrasti minimum 4.5:1 olacaktir.
- Tum form elemanlarinda `label` ve `aria-*` ozellikleri dogru kullanılacaktir.
- Desteklenen tarayicilar: Chrome, Edge, Firefox ve Safari'nin son 2 surumu.

- Hedef ekran: Mobil 360px genislikten yukari.

Responsive breakpoint'ler:

- Mobil: 360px - 639px
- Tablet: 640px - 1023px
- Desktop: 1024px ve uzeri
- Buyuk ekran: 1280px ve uzeri
- Public ogrenci ekranlarinda mobil-first yaklasim uygulanacaktır.
- Hoca panelinde sidebar 1024px altinda collapse olacak veya hamburger menu'ye donusecektir.

22. Test Guvenligi

Online sinav platformlarinda kritik bir noktadir.

22.1 Yeniden Giris Kontrolu

- Bir testin ayni ogrenci tarafindan tekrar tekrar cozulmesini engellemek icin telefon veya e-posta bazli tekil deneme kontrolu yapilir.
- `test_attempts` tablosunda `(test_id, student_id)` icin tekil indeks veya policy uygulanir.
- Hoca dilerse "tekrar denemeye izin ver" ayarini ileri fazda acabilir.

22.2 Cevap Manipulasyonu

- Sorularin dogru cevabi client'a gonderilmez.
- Sonuc hesaplamasi sadece server-side yapilir.
- Cevaplar server action'a gonderildiginde sunucu tarafinda dogrulanicir.

22.3 Test Suresi

- Sure varsa basladigi an `started_at` server-side kaydedilir.
- Sure dolunca server `completed_at` 'i otomatik isaretler ve cevap kabul etmez.
- Client tarafindaki geri sayim sayaci, her cevap gonderiminde server'dan kalan sureyi alarak kendini senkronize eder.
- Internet baglantisi kesildiginde client yerel zamanlayiciyi durdurmaz ancak baglanti donunce server'dan guncel sureyi alir.
- Client ve server arasindaki sure farki 5 saniyeyi asarsa client'taki sayac server suresine sifirlanicir.
- Sure dolmadan 1 dakika kala kullaniciya gorsel ve sesli uyari verilir.

22.4 Spam ve Bot Kontrolu

- Test baslatma endpoint'ine IP bazli rate limiting eklenir: ayni IP'den dakikada en fazla 5 test baslatma istegi.
- Hoca giris endpoint'ine rate limiting eklenir: ayni IP'den dakikada en fazla 10 giris denemesi.
- OTP gonderme endpoint'ine rate limiting eklenir: ayni telefon/e-posta'ya 5 dakikada en fazla 3 OTP istegi.

- Rate limiting Next.js middleware veya `upstash/ratelimit` kutuphanesi ile uygulanacaktır.
- İleri fazda Cloudflare Turnstile veya hCaptcha ile bot korumasına geçilecektir.

23. İçerik Sayfaları

Public alanda olması gereken içerik sayfaları:

- Hakkımızda
- İletişim
- Sıkça Sorulan Sorular (SSS)
- Gizlilik Politikası
- KVKK Aydınlatma Metni
- Kullanım Kosulları

Sorumluluk:

- Emir: Sayfa yapısı, içerik metinlerinin yerleştirilmesi, tasarım.
- Arda: Routing ve metadata, gerektiğinde admin paneli üzerinden düzenlenebilir hale getirme (ileri faz).

24. Backend Alternatifleri (Arastırma Özeti)

Bu bölüm, ana kararı vermeden önce değerlendirilen backend seçeneklerinin özeti. Seçilen yapı her zaman Bölüm 6'dadır.

24.1 Seçilen Yapı: Next.js + Prisma + PostgreSQL

Bu proje için ana yapı olarak seçilmiştir.

Kullanılacak yapı:

- Frontend: Next.js App Router
- Backend: Next.js server actions ve route handlers
- ORM: Prisma
- Veritabanı: PostgreSQL
- Veritabanı hosting: Neon
- Auth: Auth.js (NextAuth)
- Deployment: Vercel

Neden uygun:

- Proje ilişkisel veri ağırlıklı olduğu için PostgreSQL uygundur.
- Prisma migration ve type-safe sorgu desteği verir.
- Ayrıca bir backend deposu gerektirmez.
- İki kişilik ekip için geliştirme hızı yüksektir.
- Vercel preview deployment akisiyle uyumludur.

Dikkat edilmesi gerekenler:

- Veritabanı seviyesinde RLS olmadığı için yetki kontrolleri server-side kodda dikkatli yazılmalıdır.
- Neon connection pooling ayarları Vercel serverless için uygun yapılmalıdır.
- Auth.js Prisma adaptörü ile `User` , `Account` , `Session` ve `VerificationToken` tabloları otomatik oluşturulacaktır.

24.2 Firebase

Artları:

- Auth, database, hosting ve storage tarafında olgun servisler sunar.
- Hızlı prototip çıkarmak için uygundur.
- Realtime özellikleri güçlüdür.

Eksileri:

- Firestore NoSQL olduğu için hoca, öğrenci, test, soru ve sonuç ilişkileri PostgreSQL kadar rahat modellenmez.
- Maliyet büyüdükçe ongorulmesi zor olabilir.
- Vendor lock-in riski daha yüksektir.

Karar:

- Bu proje için kullanılabilir ama ana tercih değildir.

24.3 Appwrite

Artları:

- Auth, database, storage ve functions tek platformda gelir.
- Self-host edilebilir.
- Supabase benzeri bir backend deneyimi sunar.

Eksileri:

- Ekosistem ve kaynak sayısı Firebase/Supabase kadar geniş değildir.
- Veritabanı ve sorgu esnekliği PostgreSQL + Prisma kadar rahat olmayabilir.

Karar:

- Supabase benzeri hazır backend istenirse değerlendirilebilir.

24.4 PocketBase

Artları:

- Tek binary ile çok hızlı kurulur.
- Dahili admin paneli, auth ve SQLite veritabanı vardır.
- Demo ve MVP için çok pratiktir.

Eksileri:

- SQLite write concurrency nedeniyle buyuyen sistemlerde sinirli kalabilir.
- Yatay olceklemeye icin uygun degildir.
- Uzun vadeli ciddi urun icin risklidir.

Karar:

- Sadece demo veya cok kucuk MVP icin dusunulebilir.

24.5 NestJS + Prisma + PostgreSQL

Artlari:

- Ayri backend mimarisi icin profesyonel ve moduler bir secenektir.
- RBAC, servis katmani, API dokumantasyonu ve test yapisi gucludur.
- Buyuk projeye daha rahat evrilebilir.

Eksileri:

- Iki kisilik ekip icin kurulum ve gelistirme yuku artar.
- Ayrica backend deployment, API sozlesmeleri ve ekstra repo/klasor yonetimi gerekir.

Karar:

- Ilk MVP icin gerekli degildir. Proje buyurse ileride ayrilmis backend olarak dusunulebilir.

24.6 Laravel + PostgreSQL

Artlari:

- Auth, policy, middleware ve admin panel ekosistemi gucludur.
- Laravel Sanctum ile API auth kolay kurulur.
- Spatie Permission gibi paketlerle rol/yetki sistemi hizli kurulabilir.

Eksileri:

- Next.js frontend ile birlikte iki ayri teknoloji ekosistemi yonetmek gerekir.
- Ekip PHP/Laravel tarafinda deneyimli degilse gelistirme hizi duser.

Karar:

- Laravel bilen ekipler icin guclu bir secenektir; bu proje icin ana tercih degildir.

24.7 Genel Sira

Bu proje icin alternatif backend siralamasi:

1. Next.js + Prisma + PostgreSQL + Auth.js (NextAuth) - secilen yapi
2. Appwrite

3. Firebase
4. NestJS + Prisma + PostgreSQL
5. Laravel + PostgreSQL
6. PocketBase